# Estruturas de Dados Básicas II

Prof. Dr. Matheus da Silva Menezes



UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

# Conteúdo Programático




## 1 Unidade

**COMPLEXIDADE COMPUTACIONAL**

1. **Complexidade Assintótica**
   a. BIG O
   b. Theta
   c. Ômega
2. **Recorrências**
   a. Substituição
   b. Iteração
   c. Árvore de Recorrência
   d. Teorema Mestre
3. **Análise de Algoritmos**

## 2 Unidade

**ÁRVORES**

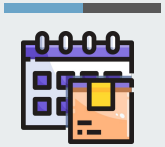
1. **Conceitos Iniciais**
2. **Árvores Binárias**
   a. Busca/Inserção/Remoção
3. **HeapSort**
   a. Busca/Inserção/Remoção
4. **Árvore AVL**
   a. Busca/Inserção/Remoção
5. **Árvore Rubro-Negra**
   a. Busca/Inserção/Remoção

## 3 Unidade

**+ ÁRVORES**

1. **Árvore B**
   a. Busca
   b. Inserção
   c. Remoção
2. **Árvore Digital**
   a. Busca
   b. Inserção
   c. Remoção
3. **Conjuntos Disjuntos**

# Cronograma



| | |
|---|---|
| 31/10/24 | Avaliação 01 |
| 19/12/24 | Avaliação 02 |
| 21/01/25 | Avaliação 03 |
| 28/01/25 | Reposição |
| 30/01/25 | Final |

## 1 Unidade

### COMPLEXIDADE COMPUTACIONAL

1. **Complexidade Assintótica**
   a. BIG O
   b. Theta
   c. Ômega
2. **Recorrências**
   a. Substituição
   b. Iteração
   c. Árvore de Recorrência
   d. Teorema Mestre
3. **Análise de Algoritmos**

**SETEMBRO**

| D | S | T | Q | Q | S | S |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | ~~17~~ | 18 | ~~19~~ | 20 | 21 |
| 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 29 | 30 | | | | | |

**OUTUBRO**

| D | S | T | Q | Q | S | S |
|---|---|---|---|---|---|---|
| | | 1 | 2 | 3 | 4 | 5 |
| 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 | 17 | 18 | 19 |
| 20 | 21 | 22 | 23 | 24 | 25 | 26 |
| 27 | 28 | 29 | 30 | 31 | | |

## 2 Unidade

### ÁRVORES

1. **Conceitos Iniciais**
2. Árvores Binárias
   a. Busca/Inserção/Remoção
3. HeapSort
   a. Busca/Inserção/Remoção
4. Árvore AVL
   a. Busca/Inserção/Remoção
5. Árvore Rubro-Negra
   a. Busca/Inserção/Remoção

**NOVEMBRO**

| D | S | T | Q | Q | S | S |
|---|---|---|---|---|---|---|
| | | | | | 1 | 2 |
| 3 | 4 | ~~5~~ | 6 | ~~7~~ | 8 | 9 |
| 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 | ~~21~~ | 22 | 23 |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 |

**DEZEMBRO**

| D | S | T | Q | Q | S | S |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 | 20 | 21 |
| 22 | 23 | ~~24~~ | 25 | ~~26~~ | 27 | 28 |
| 29 | 30 | ~~31~~ | | | | |

## 3 Unidade

### + ÁRVORES

1. Árvore B
   a. Busca
   b. Inserção
   c. Remoção
2. Árvore Digital
   a. Busca
   b. Inserção
   c. Remoção
3. Conjuntos Disjuntos

**JANEIRO**

| D | S | T | Q | Q | S | S |
|---|---|---|---|---|---|---|
| | | | 1 | ~~2~~ | 3 | 4 |
| 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 31 | |

# Onde me encontrar...







**DIMAp**

**Sala 213**

**e-mail**

**matheus.menezes@ufrn.br**

# E aí, vamos começar?

# Estruturas de Dados Básicas II

**COMPLEXIDADE COMPUTACIONAL**

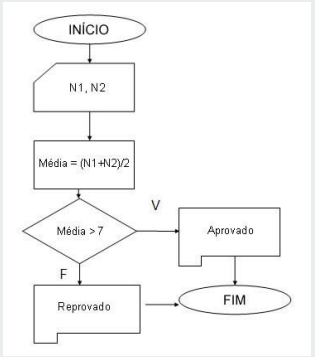
# Algumas reflexões iniciais...

**ALGORITMO**

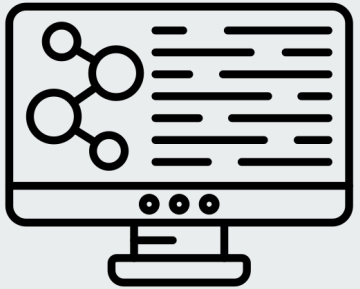
"Uma sequência de instruções (passos) não ambíguas para resolver um problema, isto é, para obter uma saída requerida para uma entrada legítima, em um período de tempo finito." (Levitin, 2012)



PROBLEMA




ALGORITMO



COMPUTADOR

# Algumas reflexões iniciais...

PSEUDOCÓDIGO

```
Algoritmo1(n)
  para i = 1 até n
    escreva("Hoje é dia de aula")
```

Qual o nome do algoritmo?

Qual o parâmetro de entrada?

Quais comandos estão sendo executados?

Qual a saída do programa?



# Algumas reflexões iniciais...

**Problema**: Soma de dois números inteiros
**Entrada**: (“a”,”b”) números inteiros
**Saída**: um número “c”, inteiro, que é a soma dos números da entrada

PSEUDOCÓDIGO

```
somaInteiros(a,b)
  c = a + b
  retorne c
```

# Algumas reflexões iniciais...

**Problema:** **Busca em lista**

**Entrada:** Uma sequência de **n** números **A={a1, ...., an}** e uma chave de busca **x**.

**Saída:** O índice **i** da sequência onde a chave **x** se localiza (**A[i] = x**)ou a indicação que a chave não se encontra na lista

**Problema:** **Ordenação de lista**

**Entrada:** Uma sequência de **n** números **A={a1, ...., an}**

**Saída:** Uma permutação (reordenação) da sequência de entrada, tal que:
**a1 ≤ a2 ≤ ... an-1 ≤ an**



# Algumas reflexões iniciais...

**INSTÂNCIA**

Ao se fixar os parâmetros de um problema elabora-se um caso concreto ou **instância**.

**Problema**: Soma de dois números inteiros
**Entrada**: (“a”,”b”) números inteiros
**Saída**: um número “c”, inteiro, que é a soma dos números da entrada



**INSTÂNCIA**

**Entrada**:
a = 3
b = 2

# Algumas reflexões iniciais...

**Problema:** Busca em lista

**Entrada:** Uma sequência de $n$ números $A=\{a1, ...., an\}$ e uma chave de busca $x$.

**Saída:** O índice $i$ da sequência onde a chave $x$ se localiza ($a(i) = x$)ou a indicação que a chave não se encontra na lista

**INSTÂNCIA**

Entrada: $n = 10$, $A=\{56, 12, 34, 8, 10, 4, 101, 37, 9, 41\}$, $x = 53$

# Algumas reflexões iniciais...

**Problema:** Ordenação de lista

**Entrada:** Uma sequência de $n$ números $A=\{a1, ...., an\}$

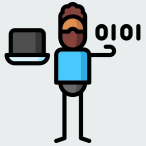
**Saída:** Uma permutação (reordenação) da sequência de entrada, tal que: $a1 \leq a2 \leq ... \ an-1 \leq an$

**INSTÂNCIA**

Entrada: $n = 10$, $A=\{56, 12, 34, 8, 10, 4, 101, 37, 9, 41\}$

**O Algoritmo deve ser capaz de resolver qualquer instância do problema.**

# Etapas para resolver o problema




1. Compreender o Problema
2. Decidir que técnica vai utilizar para desenvolver o algoritmo (exato, aproximado)
3. Desenvolver o algoritmo
4. Provar que o algoritmo está correto
5. Analisar o algoritmo
6. Codificar o algoritmo

# Algumas reflexões iniciais...

## Porque estudar complexidade de algoritmos?

1. Eficiência e Eficácia
2. Adequação à capacidade de processamento do equipamento
3. Viabilidade do código

**E se..**
*Implementar o algoritmo, aplicar a várias instâncias do problema e medir o tempo?*

## Do que depende o tempo de processamento de um algoritmo?

1. Equipamento utilizado
2. Tamanho do Problema
3. Tipo de Estrutura de dados
4. Compilador/Linguagem/SO
5. Habilidade do programador

[ ... ]

**Problema:** **Busca em lista**

**Entrada:** Uma sequência de **n** números **A={a1, ...., an}** e uma chave de busca **x**.

**Saída:** O índice **i** da sequência onde a chave **x** se localiza (**A[i] = x**)ou a indicação que a chave não se encontra na lista

PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

**Ideia:**

- Percorrer o vetor **A**, analisando cada posição **i**.
- Se **A[i]** é igual à chave procurada, retorne **i**.
- Se percorreu o vetor todo e não encontrou a chave, retorne **-1**



PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

**n = 10 x = 9**

| 56 | 12 | 34 | 8 | 10 | 9 | 101 | 37 | 4 | 41 |
|---|---|---|---|---|---|---|---|---|---|

PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

**n = 10 x = 53**

| 56 | 12 | 34 | 8 | 10 | 4 | 101 | 37 | 9 | 41 |
|---|---|---|---|---|---|---|---|---|---|

PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

`n = 10 x = 9`

| 4 | 8 | 9 | 10 | 12 | 34 | 37 | 41 | 56 | 101 |
|---|---|---|---|---|---|---|---|---|---|

**Problema:** **Busca em lista ordenada**

**Entrada:** Uma sequência de **n** números **A={a1, ...., an}** ordenados e uma chave de busca **x**. (ou seja a1 ≤ a2 ≤ ... an-1 ≤ an)

**Saída:** O índice **i** da sequência onde a chave **x** se localiza (**A[i] = x**)ou a indicação que a chave não se encontra na lista

PSEUDOCÓDIGO

```
buscaLista(A,n,x)
i = 1
enquanto i<=n e A[i]!= x faça
    i = i+1
se i <= n e A[i]==x então
   retorne i
retorne -1
```

**Ideia:**

- Percorrer o vetor **A**, analisando cada posição **i**.
- Se **A[i]** é igual à chave procurada, retorne **i**.
- Se percorreu o vetor todo e não encontrou a chave, retorne **-1**



**Problema:** **Busca em lista ordenada**

**Entrada:** Uma sequência de **n** números **A={a1, ...., an}** ordenados e uma chave de busca **x**. (ou seja a1 ≤ a2 ≤ ... an-1 ≤ an)

**Saída:** O índice **i** da sequência onde a chave **x** se localiza (**A[i] = x**)ou a indicação que a chave não se encontra na lista

PSEUDOCÓDIGO

```
buscaLista2(A,n,x)
i = 1
enquanto i<=n e A[i]< x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```




**Ideia 2:**

- Percorrer o vetor **A**, analisando cada posição **i**, enquanto **A[i]< x**
- Se **A[i]** é igual à chave procurada, retorne **i**.
- Se percorreu o vetor todo e não encontrou a chave, retorne **-1**

PSEUDOCÓDIGO

```
buscaLista2(A,n,x)
i = 1
enquanto i<=n e A[i]< x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

**n = 10 x = 9**

| 4 | 8 | 9 | 10 | 12 | 34 | 37 | 41 | 56 | 101 |
|---|---|---|---|---|---|---|---|---|---|

PSEUDOCÓDIGO

```
buscaLista2(A,n,x)
i = 1
enquanto i<=n e A[i]< x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

**n = 10 x = 11**

| 4 | 8 | 9 | 10 | 12 | 34 | 37 | 41 | 56 | 101 |
|---|---|---|---|---|---|---|---|---|---|

PSEUDOCÓDIGO

```
buscaLista2(A,n,x)
i = 1
enquanto i<=n e A[i]< x faça
    i = i+1
se i <= n e A[i]==x então
    retorne i
retorne -1
```

**n = 10 x = 125**

| 4 | 8 | 9 | 10 | 12 | 34 | 37 | 41 | 56 | 101 |
|---|---|---|---|---|---|---|---|---|---|

## Problema: Busca em lista ordenada

`n = 10 x = 9`

| 4 | 8 | 9 | 10 | 12 | 34 | 37 | 41 | 56 | 101 |
|---|---|---|---|---|---|---|---|---|---|

**Ideia 3 (Busca Binária):**
Compare a chave com a posição `A[n/2]`.
Podemos ter 3 casos:

- `A[n/2]==x`.
- Se `A[n/2]>x` a chave (caso exista), está em `A[1,...n/2]` .
- Se `A[n/2]<x` a chave (caso exista), está em `A[(n/2)+1,...n]` .

**Problema:** Busca em lista ordenada

PSEUDOCÓDIGO

```
buscaBinaria(A,n,x)
esq = 1
dir = n
enquanto esq<dir faça
    m = ⌊(esq+dir)/2⌋
    se k > A[m] então
        esq = m+1
    senão
        dir = m
se A[esq]==x então
    retorne esq
retorne -1
```

`n = 10 x = 9`

| 4 | 8 | 9 | 10 | 12 | 34 | 37 | 41 | 56 | 101 |
|---|---|---|---|---|---|---|---|---|---|

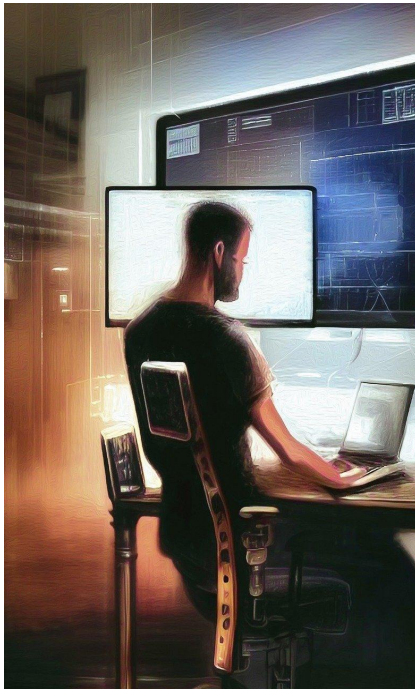
# Concluindo...

Os algoritmos resolvem o problema?

Qual dos algoritmos é o melhor?

Qual devo usar?



# Dúvidas?!




Obrigado!